

Databricks Apps bereitstellen

(Deployment)

Wie eine App im Workspace verfügbar gemacht wird – über die Oberfläche, die Kommandozeile oder aus einem Git-Repository.

"Deployment" bedeutet, eine App im Databricks-Workspace verfügbar und lauffähig zu machen: Der Code wird gebaut, Abhängigkeiten werden installiert und die App wird gestartet – basierend auf der Projektkonfiguration. Apps, die aus einer Vorlage erstellt wurden, werden bei der Erstellung automatisch bereitgestellt, können danach aber jederzeit neu bereitgestellt werden, wenn sich der Code ändert.

Wie der Build-Prozess funktioniert

Databricks unterstützt Python, Node.js oder eine Kombination aus beidem. Der Build-Prozess prüft zunächst, ob eine `package.json`-Datei vorhanden ist, um zu entscheiden, ob Node.js verwendet wird.

Wenn eine `package.json` vorhanden ist

1. Node-Abhängigkeiten installieren: über `pnpm install --frozen-lockfile` (falls `pnpm-lock.yaml` existiert) oder über `npm install`
2. Python-Abhängigkeiten installieren: aus `requirements.txt` oder über `uv sync` (falls `pyproject.toml` und `uv.lock` vorhanden sind)
3. Das `build`-Skript ausführen, falls definiert
4. Den Befehl aus `app.yaml` ausführen, oder standardmäßig `npm run start`

Wichtig: Bei npm-Apps ohne festgelegten Befehl führt Databricks immer `npm run start` aus – selbst wenn zusätzlich Python-Code vorhanden ist. Um Node- und Python-Prozesse gleichzeitig laufen zu lassen, sollte man ein eigenes `start`-Skript definieren, das ein Werkzeug wie `concurrently` nutzt.

Wenn keine `package.json` vorhanden ist

1. Python-Abhängigkeiten installieren: aus `requirements.txt` oder über `uv sync`
2. Den Befehl aus `app.yaml` ausführen, oder standardmäßig `python <my-app>.py`

Vor dem Deployment prüfen

Vor dem ersten Deployment sollte das Projekt folgendes enthalten:

- **Hauptskript** (Einstiegspunkt wie `app.py` oder `app.js`)
- **Optional eine** `app.yaml` (für eigene Befehle oder Umgebungsvariablen)
- **Abhängigkeiten**, korrekt hinterlegt in `requirements.txt` , `package.json` oder `pyproject.toml`
- **Secrets/Umgebungswerte**, die in `app.yaml` referenziert werden, müssen zugänglich sein
- **Zugriff des Service Principals** auf den Ordner mit dem Quellcode

Zwei Deployment-Quellen

Apps können aus zwei Quellen bereitgestellt werden:

- **Workspace-Ordner:** App-Dateien werden in den Workspace hochgeladen und von dort bereitgestellt (Standardmethode)
- **Git-Repository:** Ein Git-Repository wird mit der App verknüpft, die App liest bei jedem Deployment den Code direkt aus der konfigurierten Git-Referenz (Branch, Tag oder Commit) – ganz ohne Hochladen von Workspace-Dateien

Man kann jederzeit zwischen beiden Quellen wechseln.

Deployment über die Databricks-Oberfläche (Workspace-Ordner)

1. App-Dateien über die Import-Funktion in den Workspace hochladen
2. Im App-Switcher "Databricks Apps" auswählen
3. Die eigene App in der Spalte "Name" auswählen
4. Auf "Deploy" klicken und den Workspace-Ordner mit den App-Dateien wählen
5. "Select" und dann "Deploy" klicken

Deployment über die Databricks CLI (Workspace-Ordner)

1. Im Terminal in das Verzeichnis mit den App-Dateien wechseln
2. Dateien über den Sync-Befehl in den Workspace hochladen:

```
databricks sync --watch . /Workspace/Users/my-email@org.com/my-app
```

Die Option `--watch` hält die Synchronisation aktiv und lädt Änderungen automatisch hoch. Dateien lassen sich über eine `.gitignore` ausschließen (häufig: `node_modules/`, `.env`, `__pycache__`, `.DS_Store`).

3. Den Upload im Workspace-Dateibrowser überprüfen
4. Deployment starten:

```
databricks apps deploy my-app-name \  
  --source-code-path /Workspace/Users/my-email@org.com/my-app
```

Die CLI zeigt den Fortschritt an und bestätigt, sobald die App läuft.

Deployment über die Databricks-Oberfläche (Git-Repository)

1. App-Dateien in ein Git-Repository hochladen
2. Im App-Switcher "Databricks Apps" auswählen
3. Bestehende App auswählen oder neue individuelle App erstellen
4. Im Schritt "Configure Git" die Repository-URL eingeben und den Anbieter auswählen
5. Die Git-Referenz eingeben (Branch, Tag oder Commit)
6. Bei GitHub/Azure-DevOps-Branches optional "Auto deploy on push events" aktivieren
7. Auf "Create app" oder "Save" klicken
8. Bei privaten Repositories: auf der App-Übersichtsseite "Configure Git credential" klicken (erfordert `CAN MANAGE`-Berechtigung)

Anschließend erfolgt das Deployment:

1. Auf der App-Übersichtsseite auf "Deploy" klicken
2. "From Git" auswählen
3. Git-Referenz eingeben (Branchname, Tag oder Commit-SHA)
4. Referenztyp angeben (Branch, Tag oder Commit)
5. (Optional) Quellcode-Pfad für ein Unterverzeichnis angeben
6. (Optional) "Auto deploy on push events" für automatisches erneutes Deployment bei jedem Commit aktivieren

7. Auf "Deploy" klicken

Deployment über die Databricks CLI (Git-Repository)

1. App-Dateien in ein Git-Repository hochladen
2. Git-Repository beim Erstellen der App konfigurieren oder zu einer bestehenden App hinzufügen. Unterstützte Anbieter: `github`, `githubEnterprise`, `gitLab`, `gitLabEnterpriseEdition`, `bitbucketCloud`, `bitbucketServer`, `azureDevOpsServices`, `awsCodeCommit`

Neue App mit konfiguriertem Git erstellen:

```
databricks apps create my-app \  
  --json '{"git_repository": {"url": "https://github.com/org/repo", "provider": "github"}}
```

Git-Repository zu bestehender App hinzufügen/aktualisieren:

```
databricks apps create-update my-app \  
  --json '{"update_mask": "git_repository", "git_repository": {"url": "https://github.com/org/repo", "provider": "github"}}
```

Für private Repositories muss ein Git-Zugangsdatum (Credential) für das Service Principal der App konfiguriert werden (erfordert `CAN MANAGE` -Berechtigung):

```
databricks git-credentials create --json '{  
  "git_provider": "github",  
  "git_email": "your-email@example.com",  
  "personal_access_token": "YOUR_TOKEN",  
  "principal_id": YOUR_SP_ID,  
  "name": "GitHub credentials for SP"}'
```

`YOUR_SP_ID` ist dabei durch die ID des Service Principals zu ersetzen.

Deployment mit Angabe der Git-Referenz (die folgenden Optionen schließen sich gegenseitig aus):

```
# Branch  
databricks apps deploy my-app \  
  --json '{"git_source": {"branch": "main"}}'  
  
# Tag  
databricks apps deploy my-app \  
  --json '{"git_source": {"tag": "v1.0.0"}}'
```

```
# Commit
databricks apps deploy my-app \
  --json '{"git_source": {"commit": "abc123def456"}}'

# Unterverzeichnis
databricks apps deploy my-app \
  --json '{"git_source": {"branch": "main", "source_code_path": "apps/my-app"}}'
```

Wichtig: Bei Branch- oder Tag-Referenzen stellt Databricks jeweils den neuesten Commit bereit. Bei Angabe eines Commit-SHA wird genau dieser Commit bereitgestellt. Ungültige oder abgelaufene Git-Zugangsdaten führen zu fehlgeschlagenen Deployments.

Apps, die vor Einführung des Git-Deployments erstellt wurden, gewähren ihren Erstellern nicht automatisch **CAN MANAGE**-Rechte auf das Service Principal. Workspace-Administratoren müssen diese Berechtigung ggf. manuell vergeben. Jedes Service Principal unterstützt nur ein Git-Zugangsdatum pro Anbieter – eine Aktualisierung an anderer Stelle ersetzt die bestehenden Zugangsdaten.

Automatische Git-Deployments (Beta)

Diese Funktion befindet sich in der Beta-Phase und ist nur für GitHub und Azure DevOps verfügbar. Sie registriert einen Webhook im Git-Repository und stellt die App bei jedem neuen Commit auf dem konfigurierten Branch automatisch erneut bereit – so bleibt die App stets mit dem Repository synchron.

Voraussetzungen für GitHub

- Die Databricks-GitHub-App muss im Repository installiert sein
- Das Repository muss privat sein (öffentliche Repositories werden nicht unterstützt)
- Das Service Principal benötigt ein Git-Zugangsdatum mit Repository-Zugriff

Voraussetzungen für Azure DevOps

- Der Nutzer, der automatische Deployments aktiviert, benötigt ein konfiguriertes Git-Zugangsdatum
- Dieses Zugangsdatum benötigt die Berechtigung "Edit subscriptions" für Service Hooks

Aktivieren lässt sich die Funktion im Schritt "Configure Git" beim Erstellen einer individuellen App oder über das "Auto-deployment"-Badge einer bestehenden App bzw. in

den App-Einstellungen. Die Git-Referenz muss dabei ein Branchname sein – Tags sind mit automatischen Deployments nicht kompatibel. Deaktivieren lässt sich die Funktion über das "Auto-deployment"-Badge.

Nur Git-Deployments erzwingen

Workspace-Administratoren können festlegen, dass alle Apps ausschließlich aus Git bereitgestellt werden dürfen: unter "Settings" > "Development" > "Apps" die Option "Only allow app deployments from Git" aktivieren (standardmäßig deaktiviert). Ist diese Option aktiv:

- Nutzer müssen vor dem Erstellen einer App ein Git-Repository konfigurieren
- Deployment ist nur noch aus Git möglich
- App-Vorlagen sind deaktiviert
- Git-Repositories können nicht mehr von Apps entfernt werden
- Bestehende Apps laufen weiter, können aber ohne Git-Repository nicht mehr neu bereitgestellt werden

Verhalten nach dem Deployment

Nach dem Deployment startet Databricks die App mit dem `command` aus `app.yaml`, oder standardmäßig mit `python app.py`. Die App-Übersichtsseite zeigt aktuellen Status, Logs, Deployment-Historie und Umgebungsinformationen. Über den App-Link gelangt man zur laufenden App, im Reiter "Logs" lassen sich Fehler debuggen und die Laufzeit überwachen.

Erneutes Deployment (Redeployment)

Nach Änderungen am Quellcode oder an der Konfiguration lässt sich die App erneut bereitstellen, ohne sie neu zu erstellen. Ein Wechsel zwischen Workspace- und Git-Quelle ist jederzeit möglich.

Aus dem Workspace-Ordner

1. App-Dateien im Workspace aktualisieren
2. App auswählen und auf "Deploy" klicken
3. Bei geändertem Quellpfad oder Wechsel von Git: Pfeil neben "Deploy" anklicken und "Deploy using a different source" wählen

Aus dem Git-Repository

1. Änderungen ins Repository pushen
2. App auswählen und auf "Deploy" klicken
3. Bei geänderter Git-Referenz oder Wechsel vom Workspace: Pfeil neben "Deploy" anklicken und "Deploy using a different source" wählen

Wichtig: Ein Wechsel des Git-Repositorys oder der Deployment-Quelle (Git/Workspace) löscht alle Git-Zugangsdaten des Service Principals. Wird nur die Git-Referenz geändert, bleiben die Zugangsdaten erhalten. Vor einem erneuten Git-Deployment müssen die Zugangsdaten dann neu konfiguriert werden.

Fehlerbehebung beim Deployment

Falls das Deployment fehlschlägt oder die App nicht wie erwartet läuft, helfen folgende Prüfungen:

- Logs auf Fehlermeldungen und Laufzeitausgaben prüfen
- Syntax und Einstellungen der `app.yaml` validieren
- Prüfen, ob Secrets und Umgebungsvariablen im Abschnitt `env` korrekt aufgelöst werden
- Sicherstellen, dass alle benötigten Abhängigkeiten enthalten bzw. installierbar sind
- Bei Private Link oder eingeschränkten Egress-Richtlinien prüfen, ob die benötigten Domains freigegeben sind (fehlende Freigaben sind eine häufige Ursache für Deployment-Fehler bei Private Link)

Speziell bei Git-Deployments:

- Bei privaten Repositories prüfen, ob das Service Principal ein Git-Zugangsdatum besitzt
- Repository-URL und Git-Referenz (Branch, Tag oder Commit) auf Korrektheit prüfen
- Falls Git-Only erzwungen wird, können Apps ohne konfiguriertes Git-Repository nicht starten
- Bei Deployment über CLI/API/Declarative Automation Bundles: zuerst die App erstellen, danach das Git-Zugangsdatum zum Service Principal hinzufügen

Quelle: <https://docs.databricks.com/aws/en/dev-tools/databricks-apps/deploy>

Stand: 2026-08-06